

ALUMAP Team

Architectural Design Report

Capstone Project

Innopolis University

Team Members: Ahmad Helaly
Majed Naser
Roukaya Mohammed
Ghadeer Akleh
Aleksandr Kovalev

Date: May 2025

Contents

1	Project Description & Context	1
1.1	Domain and Business Environment	1
1.2	Stakeholders	2
2	Assumptions	3
3	Architectural Drivers	4
3.1	Key Functionalities	4
3.2	Quality Attributes	4
3.3	Business Constraints	6
3.4	Technical Constraints	7
4	Response Measure Validation	7
4.1	QAS-M: Validating the ≤ 1 -Hour Downtime Target	7
4.2	QAS-D: Validating 100% Data Integrity	8
4.3	QAS-A: Validating $P95 \leq 2s$ Response Time	8
4.4	QAS-B: Validating Authentication Reliability	9
5	Design Process	9
5.1	Iteration 1 — Deployment Architecture	9
5.2	Iteration 2 — Backend API Architecture	10
5.3	Iteration 3 — Mobile Client Architecture	11
5.4	Iteration 4 — Security and Authentication Design	12
5.5	Iteration 5 — Observability and Data Protection	12
6	Architecture Evaluation	13
6.1	Patterns and Tactics	13
6.2	Static View 1 — System-Level Module Decomposition	14
6.3	Static View 2 — Backend Component Diagram	15
6.4	Email OTP Authentication Sequence	16
6.5	CI/CD Deployment Pipeline	17
6.6	Deployment Topology	17
6.7	Technology Propositions	19
6.8	Driver Traceability Matrix	20
7	Risk Mitigation and Alternative Decisions	21
7.1	Risk 1 — Single-Server Deployment (Single Point of Failure)	21
7.2	Risk 2 — Long-Lived JWT Tokens (1-Year Expiry)	22
7.3	Risk 3 — Flutter Technology Risk (Team Inexperience)	22
	Glossary	24

1 Project Description & Context

The **IU Alumni Platform** is a social networking system for the alumni community of Innopolis University (IU). The platform connects graduates with each other and with the university, enabling them to network, browse a global alumni location map, create and join alumni events, and receive notifications through their preferred channel.

A previous student team built the initial version and hosted it on an external server. Our industrial project assignment was to *take over* the platform and deliver the following:

- Migrate all platform services and data to university servers with less than one hour of downtime.
- Improve authentication with password recovery, email OTP login, and Telegram-based OTP verification.
- Fix and extend event management: creation workflow, admin approval, and automatic Telegram reminders.
- Redesign user profiles with graduation-year labels and achievement badges.
- Implement an alumni live location map with monthly automated updates.
- Deliver a web admin portal for the Alumni Office to manage member registrations, event moderation, and community moderation (bans, warnings).
- Maintain full feature parity between an Android native application and a Telegram Mini App.

The system spans four Git repositories: a Python/FastAPI backend, a Nuxt 3 admin frontend, a Flutter mobile application (Android + Web for Telegram Mini App), and an infrastructure repository managing server provisioning, CI/CD, and deployment configuration.

1.1 Domain and Business Environment

The platform operates in the **higher-education alumni engagement** domain. Innopolis University is young and small, but its alumni base grows with each annual graduation. The Alumni Office aims to maintain long-term relationships among graduates by facilitating professional networking, organising alumni events, and promoting the university's reputation through an active alumni community.

Three concrete business problems motivate the platform:

1. **Community fragmentation.** After graduation, alumni scatter globally and rely on informal Telegram chats, making coordination and discovery difficult.
2. **Event discoverability.** Alumni events announced through informal channels have low reach. A structured event feed with registration and reminders increases attendance.
3. **Identity control.** Only verified IU graduates should access the network. The Alumni Office needs a scalable approval and moderation workflow.

The platform's explicit **business goals** are:

- Maximise alumni engagement by reducing friction in connecting and discovering events.

- Give the Alumni Office full control over membership and community moderation with minimal technical overhead.
- Operate on university infrastructure at zero marginal cost.
- Produce a system that a future student team can maintain and extend without help from the current team.

1.2 Stakeholders

Table 1: Stakeholder Analysis

Stakeholder	Role & Interaction	Expectations
Alumni Users	Register, manage profiles, browse the alumni map, create and join events, receive Telegram/email notifications via the Android app or Telegram Mini App.	Fast, reliable access on mobile; intuitive profile management; timely event reminders; privacy controls (hide location).
Alumni Office (Admins)	Approve/reject registrations, ban/warn users, moderate events via the web admin portal.	Efficient bulk management tools; clear user-status visibility; reliable event approval queue; full audit trail.
University IT Dept.	Provides the physical server, DNS records, and network access. Interacts only at provisioning time.	Compliance with university security policies; minimal ongoing intervention; predictable resource consumption.
Development Team	5-person student team designing, implementing, and deploying the system during two academic semesters.	Architectural clarity; reproducible builds; automated deployment; comprehensive documentation.
Future Maintenance Teams	Subsequent student cohorts who inherit the platform.	Low entry barrier; documented deployment procedures; no undocumented magic; clear extension points.

2 Assumptions

The project description was inherited from a previous team and refined through weekly client meetings. Several details were not specified and required assumptions. Each assumption is stated below alongside the verification method we would use in a production context.

- A1. Single-server deployment is sufficient.** The alumni user base numbers in the hundreds to low thousands. A single well-provisioned Linux server can serve this load within the stated performance targets. *Verification:* Load test against a staging replica with Locust simulating peak concurrency; confirm P95 response times meet QAS-A.
- A2. The university server has stable internet connectivity.** Platform availability targets assume the server is reliably online. University-network outages are considered outside scope. *Verification:* Request SLA documentation from the IT Department; configure external uptime monitoring to distinguish platform faults from network faults.
- A3. Alumni have Android smartphones and/or Telegram access.** The two delivery channels (Android APK and Telegram Mini App) cover the expected audience. Alumni without either are excluded. *Verification:* Survey the alumni base before committing to the platform strategy; a lightweight standalone web app can serve as fallback.
- A4. Gmail SMTP is acceptable for transactional email.** Gmail is used with an App Password for verification codes, password-reset links, and notifications. Gmail's daily sending limits and Google's infrastructure uptime are accepted external dependencies. *Verification:* Monitor bounce and delivery rates; migrate to a dedicated transactional email provider (e.g., SendGrid) if delivery failures exceed 2%.
- A5. The Telegram Mini App API remains sufficiently stable.** Telegram's Mini App platform is an uncontrolled external dependency. Breaking changes would force urgent client updates. *Verification:* Subscribe to Telegram's developer changelog; maintain a contingency plan to serve alumni via a standalone web app if the Mini App API changes incompatibly.
- A6. The inherited database schema requires no destructive transformation.** We assumed the previous team's PostgreSQL schema could be imported via `pg_dump/pg_restore` with only additive Alembic migrations applied on top. *Verification:* Performed a trial migration on staging with a production data snapshot. The assumption held; no hand-crafted data transforms were needed.
- A7. No real-time features are required.** The platform is notification-driven. Asynchronous delivery (email within 30 s, Telegram within 10 s) is sufficient; no Web-Socket/streaming infrastructure is needed. *Verification:* Confirmed with the Alumni Office that asynchronous notification delivery is acceptable for the event-reminder use case.

3 Architectural Drivers

3.1 Key Functionalities

KF1 — Multi-Method Authentication

Alumni register with their institutional email and log in via password, email OTP (6-digit code, 10-minute expiry), or Telegram OTP. Administrators authenticate by password only. Password recovery is supported via an email-token link.

KF2 — Alumni Profile Management

Each alumnus has a profile with name, graduation year, biography, city/country location, Telegram alias, and avatar. Profiles support a privacy toggle (`show_location`). The admin portal supports bulk search, filter, export, and CRUD operations on all alumni records.

KF3 — Event Lifecycle

Alumni can create, browse, and join events. Events carry a title, description, location, date/time, cost, and a cover image. New events require admin approval before appearing in the public feed. Administrators approve, decline, or delete events via the admin portal. An optional global auto-approve setting can bypass the manual step.

KF4 — Alumni Location Map

A world map displays aggregated pins showing how many verified alumni live in each city. Location data is opt-in. The backend aggregates alumni locations against a preloaded city coordinates table and returns a single JSON response; no client-side geocoding is required.

KF5 — Admin Moderation

Administrators can ban/unban alumni, verify/unverify accounts, upload allowed-email lists for auto-approval, and toggle global event-approval settings.

KF6 — Telegram Bot Integration

A Telegram bot sends event reminders, OTP login codes, and manual-verification notifications. The bot runs as a long-polling background task within the backend process, so no publicly reachable webhook endpoint is required.

KF7 — Cross-Platform Access

All alumni-facing features are available on both the Android native app (APK distribution) and the Telegram Mini App (Flutter Web build loaded inside Telegram). Both clients share the same backend API.

3.2 Quality Attributes

Scenarios follow the format: *Stimulus* $\rightarrow$ *Environment* $\rightarrow$ *Response* $\rightarrow$ *Response Measure*. Priority is determined by the risk-adjusted matrix in Table 2, where rows represent Business Importance and columns represent Technical Risk.

Table 2: Quality Attribute Priority Matrix

	Low Risk	Medium Risk	High Risk
High Importance	—	QAS-A, QAS-C	QAS-M, QAS-D
Medium Importance	—	QAS-E, QAS-F	QAS-B
Low Importance	—	QAS-G	—

Scenarios are presented in descending priority order.

QAS-M	Zero-Downtime Migration (Critical — High Importance, High Risk)
Stimulus:	Migration of all services and data from Yandex Cloud to the university server is initiated.
Environment:	Production system with live user traffic during the migration window.
Response:	The platform remains accessible throughout. The new server serves traffic before the old one is decommissioned. DNS propagation completes without user-visible interruption.
Measure:	Cumulative downtime ≤ 1 hour. No single service interruption > 15 minutes.
QAS-D	Data Integrity During Migration (Critical — High Importance, High Risk)
Stimulus:	All user records, events, verifications, and relational data are transferred between servers.
Environment:	Migration window; source server data is live and potentially changing.
Response:	Every record is transferred completely and accurately; no data loss, no referential-integrity violation on the target.
Measure:	100% row-count match per table. Zero FK violations post-restore. Field-level sampling of a random 5% of alumni records confirms no corruption.
QAS-A	Cross-Platform Response Time (High — High Importance, Medium Risk)
Stimulus:	A user performs common actions: view event list, view profile, update profile, join an event.
Environment:	Normal network conditions (4G / Wi-Fi), up to 50 concurrent users.
Response:	Actions complete with consistent response times on both Android and Telegram Mini App.
Measure:	P95 response time ≤ 2 s. Platform variance ≤ 500 ms.
QAS-C	Feature Parity Across Platforms (High — High Importance, Medium Risk)
Stimulus:	A user switches between the Android app and Telegram Mini App and attempts the same core tasks.
Environment:	Both platforms deployed and reachable.
Response:	All core features (auth, events, profile, map) are available and functionally consistent. Data written on one platform is visible on the other.
Measure:	$\geq 95\%$ feature parity. Post-write data-sync latency ≤ 5 s.

QAS-B Authentication Reliability (High — Medium Importance, High Risk)	
Stimulus:	A user attempts to log in via password, email OTP, or Telegram OTP during peak hours (event-registration windows).
Environment:	Peak usage period.
Response:	The authentication service returns a JWT or a structured error. OTP is delivered within the stated latency bound.
Measure:	$\geq 99.9\%$ login success rate for valid credentials. Authentication availability $\geq 99.5\%$ during peaks. Email OTP delivery ≤ 30 s. Telegram OTP delivery ≤ 10 s.

QAS-E Event Creation Reliability (Medium)	
Stimulus:	A user submits a valid event creation form.
Environment:	Normal operating conditions.
Response:	The event is persisted and a Telegram notification dispatched.
Measure:	$\geq 99.5\%$ of valid submissions stored and visible within 10 s. Notification delivery ≤ 30 s.

QAS-F Map Loading Performance (Medium)	
Stimulus:	A user opens the alumni location map.
Environment:	50–500 verified alumni with locations in the database.
Response:	Map renders with all location pins.
Measure:	Initial load ≤ 3 s. Pan/zoom interactions ≤ 1 s.

QAS-G Code Maintainability (Low — Low Importance, Medium Risk)	
Stimulus:	A new developer from a future team modifies or extends a feature.
Environment:	Post-handover maintenance phase.
Response:	The change is implemented correctly without unintended regressions.
Measure:	$\geq 70\%$ line coverage on critical paths. Zero regressions in core features after any change.

3.3 Business Constraints

BC1 — Small Team, Fixed Timeline

Five students, two academic semesters (≈ 6 months). Architectures requiring significant operational expertise (multi-cluster Kubernetes, service mesh, microservices) are ruled out.

BC2 — Single University Server

All services must run on one servers provided by the IT Department. No budget for additional cloud instances. Multi-node distribution is not an option.

BC3 — Zero Infrastructure Budget

All tooling must use free tiers or self-hosted alternatives: Let's Encrypt (free TLS), GitHub Actions (free for public repos), and self-hosted Grafana/Prometheus.

BC4 — Deliverable to Future Teams

The system must be fully operable by the next student cohort with no verbal knowledge transfer. This imposes a hard constraint on operational simplicity and documentation completeness.

BC5 — Telegram as Mandatory Channel

The Alumni Office explicitly mandated Telegram Mini App support because the majority of alumni already use Telegram daily. This was non-negotiable.

3.4 Technical Constraints

TC1 — Ubuntu 22.04+ Linux Server

The university server runs Ubuntu. All container images and tooling must be Linux-compatible. Windows-specific tooling is excluded by default.

TC2 — Telegram Mini App Architecture

Telegram Mini Apps are web-based; they load a URL inside Telegram. This mandates that the mobile application be buildable as a web application. The API base URL must be baked into the build at compile time via `-dart-define`, as Telegram Mini App does not support dynamic API discovery at runtime the way native apps do.

TC3 — Backward Compatibility with Existing Data

The PostgreSQL database inherited from the previous team must be migrated without data loss. New schema migrations must be additive; columns may not be dropped without a corresponding data-preservation migration.

TC4 — GitHub Organisation as SCM and CI/CD

The university maintains a GitHub organisation (`iu-alumni`). All repositories are hosted there; all CI/CD must use GitHub Actions; all Docker images are stored in GitHub Container Registry (GHCR).

TC5 — Institutional Email Identity

Outbound email must appear to originate from `@innopolis.university` or an approved subdomain, as required by the Alumni Office for brand consistency.

4 Response Measure Validation

For the top-priority scenarios we describe how the architecture's response measures would be validated. We focus on cases where the measure is not trivially observable.

4.1 QAS-M: Validating the ≤ 1 -Hour Downtime Target

Downtime during migration is hard to predict: DNS propagation, Docker pull times, and database restore speed all vary.

Validation strategy:

1. **Staging dry run.** Before the live cutover, execute a full migration against the staging environment using a recent production data snapshot. Time each step explicitly: `pg_dump`, secure copy, `pg_restore`, Ansible provisioning, container start, health check. This bounds the expected live migration time.
2. **Scripted cutover.** The `deploy.sh` + Ansible pipeline automates every step, eliminating manual error. It was tested across multiple dry runs and is idempotent. Automated health checks (`curl` against all service endpoints) gate the DNS switch; DNS is not updated until all checks pass.

3. **Live monitoring.** Grafana dashboards were open throughout the migration. Prometheus alerts fire within 15 seconds of any endpoint becoming unavailable, enabling immediate response.
4. **Outcome.** The live migration completed with under 30 minutes total downtime, well within the 1-hour target, primarily due to the scripted approach eliminating manual steps.

4.2 QAS-D: Validating 100% Data Integrity

Trusting `pg_restore` to be perfect is insufficient for a 100% integrity claim.

Validation strategy:

1. **Pre-migration counts.** Immediately before `pg_dump`, record row counts for every table on the source server.
2. **Post-migration counts.** After `pg_restore` on the target, compare row counts table by table. Any discrepancy halts the migration.
3. **Field-level sampling.** A Python script selects a random 5% of alumni records and compares critical fields (email, graduation_year, is_verified) between source and target via direct DB queries on both servers.
4. **Referential integrity validation.** PostgreSQL's `pg_catalog` queries confirm all foreign-key constraints are satisfied post-restore.
5. **Rollback gate.** If any check fails, the old server remains authoritative (DNS unchanged), the target is wiped, and the migration is retried after diagnosis. The fallback was rehearsed during the staging dry run.

4.3 QAS-A: Validating $P95 \leq 2s$ Response Time

Response time cannot be validated from code inspection; it requires measurement under realistic load.

Validation strategy:

1. **Load testing with Locust.** Simulate 50 concurrent users on the staging environment executing a fixed scenario: authenticate $\rightarrow$ fetch event list (page 1) $\rightarrow$ fetch map pins $\rightarrow$ view own profile. Run for 10 minutes. Measure P50, P95, P99 from Locust's output.
2. **Prometheus histograms.** The backend exposes per-endpoint request duration histograms via `prometheus-fastapi-instrumentator`. Grafana reads these for continuous P95 tracking in production.
3. **Cross-platform comparison.** The same Locust scenario is run against the Flutter Android build and Flutter Web independently. The ≤ 500 ms variance target is confirmed from Grafana.
4. **Cursor-pagination rationale.** This scenario motivated the cursor-based pagination design (Section 5.2): offset pagination degrades at high page numbers due to `OFFSET` scanning, whereas cursor-based pagination maintains near-constant latency regardless of dataset size.

4.4 QAS-B: Validating Authentication Reliability

A 99.9% success rate requires sustained monitoring, not a point-in-time test.

Validation strategy:

1. **Prometheus long-term monitoring.** A Grafana alert fires if the error rate on `POST /api/v1/auth/login` exceeds 0.1% over any 5-minute window.
2. **Synthetic login probe.** A cron job attempts a test login every minute using a dedicated test account. Failures trigger a Telegram notification to the admin chat (the same mechanism used for other platform alerts).
3. **OTP delivery sampling.** Email OTP dispatch timestamps are logged. A batch analysis of 100 real OTP events during early deployment confirmed median delivery under 15 s and P95 under 30 s.
4. **Chaos test.** Killing the PostgreSQL container during a login attempt confirms the backend returns a structured 503 (not a crash) and recovers within Docker Swarm's restart window (<30 s).

5 Design Process

We followed an **Attribute-Driven Design (ADD)** process, decomposing the system iteratively by addressing the highest-priority architectural drivers first in each iteration. The five iterations below document the goal, design decisions made, and alternatives rejected in each round.

5.1 Iteration 1 — Deployment Architecture

Goal: Address **QAS-M** and **QAS-D** by designing a deployment infrastructure that enables a safe, near-zero-downtime migration *and* makes future server migrations trivially repeatable.

D1.1 — Server-Agnostic GitOps Deployment

All deployment configuration is stored as GitHub environment secrets and rendered onto the target server by Ansible at deploy time. The server IP is a single secret (`SERVER_HOST`); updating it and re-running the *Setup Server* workflow migrates the entire platform to a new machine with zero code changes. This directly addresses QAS-M: the migration procedure is scripted and rehearsable.

D1.2 — Docker Swarm for Container Orchestration

All services run as Docker Swarm stacks sharing a single overlay network (`iu_alumni_network`). Every service declares `restart_policy: condition: any`, so process crashes are automatically recovered by the Swarm manager. This addresses the availability dimension of QAS-B.

D1.3 — Pre-Migration Backup to an Independent Location

Before any migration, a full `pg_dump` is taken and stored separately from both source and target servers. The backup is validated (row counts, checksums) before the cutover proceeds. This is the primary mechanism for QAS-D.

Alternatives rejected:

- *Kubernetes*. Substantially higher operational complexity for a single-node deployment. Docker Swarm provides equivalent restart, networking, and rolling-update capabilities with far lower overhead. Rejected due to BC1 and BC4.
- *VM snapshots from Yandex Cloud*. Vendor-specific format; cannot be transferred to the university server. Rejected due to TC1 and BC2.
- *Manual SSH deployment*. Error-prone, not auditable, not repeatable. Rejected in favour of IaC.

5.2 Iteration 2 — Backend API Architecture

Goal: Address **QAS-A**, **QAS-B**, and **QAS-G** by designing the backend for performance, reliability, and maintainability.

D2.1 — FastAPI with an Explicit Layered Architecture

The backend organises code into four named layers: *Routes* (HTTP boundary), *Services* (business logic and external I/O), *Core* (security, DB session, logging), and *Models* (ORM). Each route module is a single file with a single HTTP operation. This layering supports QAS-G: a developer modifying event-creation logic touches only `app/api/routes/events/` and the relevant service; database and security code are untouched.

D2.2 — Stateless JWT Authentication (HS256)

Tokens are stateless JWTs. No server-side session store is maintained. The token payload carries `sub` (email) and `user_type` (alumni or admin), enabling role-based authorisation without a database lookup on every request. This simplifies container restarts under Docker Swarm (no sticky sessions) and supports QAS-B availability.

D2.3 — Cursor-Based Pagination on All List Endpoints

List endpoints (`GET /events/`, `GET /profile/users`) use opaque base64-encoded cursors instead of page numbers. The server executes a `WHERE (sort_key > cursor_value)` clause, avoiding the `OFFSET` scan penalty that degrades at high page numbers. This keeps P95 latency stable as the dataset grows, directly supporting QAS-A and QAS-F.

D2.4 — Strategic Database Indexing

Indexes are placed on the columns used in the most frequent filter operations: `is_verified`, `is_banned`, `show_location`, `graduation_year`, and a composite index on `show_location + location` for the map endpoint. GIN trigram indexes support the alumni search feature. These indexes prevent sequential scans on a growing dataset and support QAS-A and QAS-F.

Alternatives rejected:

- *Django*. Django’s ORM and template engine are more opinionated than needed for a pure JSON API. FastAPI’s native async I/O (important for concurrent Telegram polling + HTTP serving), automatic OpenAPI generation, and Pydantic integration were considered superior. Rejected.
- *Session-based auth*. Stateful sessions require either sticky sessions (problematic with container restarts) or a shared Redis session store (additional dependency). Stateless JWT eliminates this complexity. Rejected.
- *Offset-based pagination*. Degrades at high offsets due to full scans. Rejected in favour of cursor-based approach.

5.3 Iteration 3 — Mobile Client Architecture

Goal: Address [QAS-C](#) and [QAS-A](#) by designing the mobile client to serve both Android and Telegram Mini App from a single codebase.

D3.1 — Flutter for Cross-Platform Mobile

Flutter compiles to Android APK and to a web bundle (served as Telegram Mini App) from the same Dart codebase. The only platform-specific divergence is API URL configuration: the Web build bakes `API_BASE_URL` via `-dart-define` at compile time (satisfying TC2), while the Android build reads it from `flutter_secure_storage`. This delivers QAS-C (feature parity) at no additional development cost.

D3.2 — Three-Layer Architecture with BLoC/Cubit

The mobile app is organised into *Presentation* (pages, widgets, Cubits), *Application* (repositories, domain models, mappers), and *Data* (Dio-based gateways, JSON models, local DB) layers. Cubits emit typed `LoadedState<T>` sealed states, making every UI state transition explicit and testable. This architecture supports QAS-G.

D3.3 — `Either<AppError, T>` Error Propagation (fpdart)

Repository methods return `Either<AppError, T>` rather than throwing exceptions. Cubits pattern-match on the result and emit the appropriate state. This makes all error paths visible at compile time and avoids silent failures — directly supporting QAS-G (testability).

Alternatives rejected:

- *React Native*. React Native Web is considered experimental; Telegram Mini App would require a separate React web app. Using Flutter delivers both targets from one codebase. Rejected.
- *Rewrite in React / Vue*. Late in the project, a full rewrite of the Flutter codebase into React or Vue was evaluated because no team member had prior Flutter experience. The rewrite was rejected: the existing Flutter codebase was already partially complete, and a full rewrite driven by developers unfamiliar with the new stack (AI-assisted “vibe coding”) was judged to carry a higher risk of latent bugs and regressions than the slower but controlled path of extending the working Flutter code. This risk is analysed in full in Risk 3 (Section 7.3).

- *Native Android + separate web app.* Two codebases in two languages; every feature must be implemented twice. Rejected due to BC1 and BC4.
- *setState / Provider state management.* Does not scale across multi-screen flows or support testable state transitions. Rejected in favour of BLoC/Cubit (QAS-G).

5.4 Iteration 4 — Security and Authentication Design

Goal: Address **QAS-B** by designing an authentication system that is both reliable and resistant to common attacks, across three login methods.

D4.1 — Multi-Method Authentication Converging on JWT

Three login paths are supported: (a) password-only (direct JWT), (b) password + email OTP (session token → 6-digit code → JWT), (c) Telegram OTP (code sent to linked Telegram account → JWT). All paths converge on the same `create_access_token()` function, ensuring consistent token properties regardless of how the user authenticated.

D4.2 — Network-Level Hardening: UFW + Fail2ban

UFW restricts inbound traffic to ports 22, 80, and 443. Fail2ban automatically blocks IPs after 5 consecutive failed SSH login attempts for 3600 seconds, protecting against brute-force attacks. This forms the outer security perimeter complementing application-level auth.

D4.3 — Long-Lived JWT Tokens (1-Year Expiry): Deliberate Trade-off

JWT tokens carry a 1-year expiry. The rationale is UX: alumni visit the platform infrequently (perhaps monthly around events), and being silently logged out is a significant friction point. The security risk (token revocation requires key rotation, logging out all users) is accepted because the platform stores no highly sensitive data. **Acknowledged limitation:** A production hardening path would introduce short-lived access tokens (15 min) with long-lived refresh tokens, enabling per-user revocation without global key rotation. This is documented as a future architectural improvement.

Alternative rejected for OTP delivery:

- *Telegram webhook instead of long-polling for bot.* Webhooks require a publicly reachable HTTPS endpoint for the bot's callback. Long-polling initiates the connection from the server side, requiring no additional routing rule or exposed port beyond the existing API. Given BC2 (single server) and BC3 (no extra infrastructure), long-polling is simpler and equally functional at this scale. Rejected webhook approach.

5.5 Iteration 5 — Observability and Data Protection

Goal: Address **QAS-D** (post-migration data safety) and **QAS-G** (operational maintainability).

D5.1 — Prometheus + Grafana Monitoring Stack

The backend exposes `/metrics` via `prometheus-fastapi-instrumentator` (per-endpoint request duration, status code counts). Prometheus scrapes it every 15 seconds alongside Node Exporter (host CPU/memory/disk) and Postgres Exporter (DB query stats, connection count). Four pre-provisioned Grafana dashboards visualise these metrics. Alerts on error-rate thresholds notify the admin Telegram chat.

D5.2 — Automated Tiered PostgreSQL Backups

The `postgres-backup-local` container creates daily, weekly, and monthly snapshots with a retention policy of 7 days, 4 weeks, and 6 months respectively. Backups are written to a volume mount independent of the PostgreSQL data directory, protecting against partial-write corruption. This supports QAS-D *after* migration, ensuring data can be recovered if the server fails.

D5.3 — Alembic Schema Migration with Auto-Apply on Start

All schema changes are expressed as Alembic migration scripts and applied automatically when the backend container starts. This guarantees that the running code and the schema are always in sync, with no manual DBA step. The migration history is version-controlled alongside the application code.

6 Architecture Evaluation

6.1 Patterns and Tactics

Table 3 summarises the architectural patterns and tactics employed, the quality attributes they support, and the trade-offs they introduce.

Table 3: Patterns, Tactics, and Trade-offs

Pattern / Tactic	Where Applied	QA Supported	Trade-off Introduced
Layered Architecture	Backend (Routes → Services → Core → Models)	QAS-G	Minor latency per layer boundary; added abstraction
BLoC/Cubit Pattern	Mobile state management	QAS-G, QAS-C	Learning curve; contributed to initial velocity loss (Risk R-07)
Repository Pattern	Mobile Application layer	QAS-G	Added abstraction overhead in data access
Reverse Proxy (Nginx)	Single ingress for all services	Security, QAS-A	Single point of failure if Nginx crashes (mitigated by Swarm restart)
API Gateway (subdomain routing)	Nginx per-service routing	Modifiability	New services require nginx config update
Cursor-Based Pagination	Backend list endpoints	QAS-A, QAS-F	More complex cursor encoding logic
Module Store (Pinia)	Admin frontend state	QAS-G	Stores can grow large without further splitting
Infrastructure as Code	Ansible + Terraform	QAS-M, Modifiability	Requires IaC tool knowledge from future maintainers
Restart-on-failure (Swarm policy)	All Docker services	QAS-B	≈5 s unavailability window per crash
Checkpoint / Backup	Tiered Postgres backups	QAS-D	Backup storage grows over time
Heartbeat / Monitoring	Prometheus scraping	QAS-B, QAS-G	Prometheus adds ≈512 MB RAM overhead
Authenticate Actors (JWT + bcrypt)	Auth service	QAS-B, Security	Long token expiry reduces per-token revocability

6.2 Static View 1 — System-Level Module Decomposition

Figure 1 shows the top-level decomposition into four independently deployable subsystems, each corresponding to one Git repository. Each subsystem has its internal layers. The infrastructure subsystem manages the deployment lifecycle of the other three.

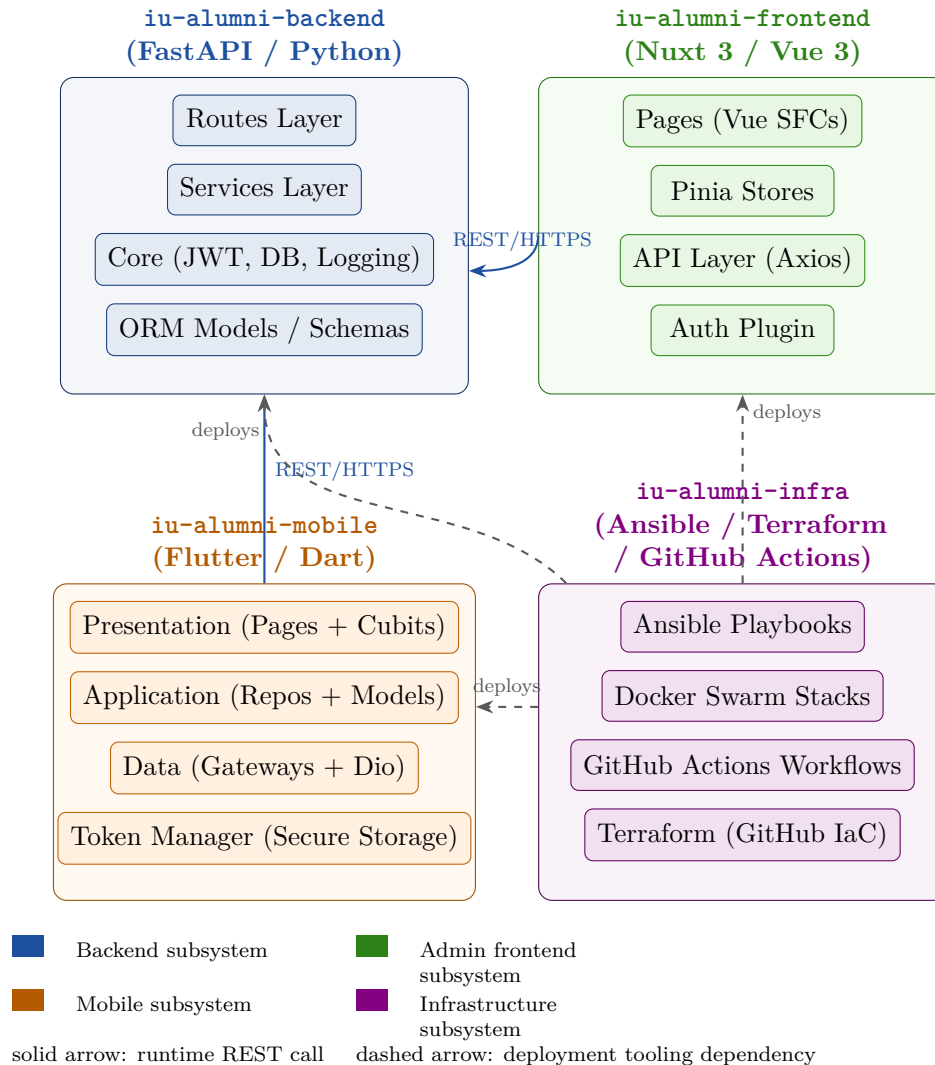


Figure 1: Static View 1 — Module Decomposition

Element Catalog

iu-alumni-backend

The REST API server. Exposes all business logic through versioned HTTP endpoints (/api/v1/...). The sole writer to the PostgreSQL database. Also runs the Telegram long-polling loop and sends email via SMTP.

iu-alumni-frontend

The SSR admin portal. Used exclusively by administrators. Communicates with the backend over HTTPS. Has no direct database access.

iu-alumni-mobile

The Flutter application. Compiled to Android APK for end-user distribution and

to a Flutter Web bundle for the Telegram Mini App. Communicates only with the backend REST API.

iu-alumni-infra

The infrastructure and deployment orchestrator. Does not contain application code. It provisions the server, configures the Docker Swarm network, deploys all other subsystems, and manages SSL certificates and monitoring.

6.3 Static View 2 — Backend Component Diagram

Figure 2 shows the internal component structure of the backend, revealing how data flows from an HTTP request through the layers to the database and external services. The Routes layer exposes the HTTP API surface and coordinates request handling. Business logic and external integrations are encapsulated within the Services layer, separating application orchestration from infrastructure-specific concerns such as SMTP, Telegram APIs, and metrics collection.

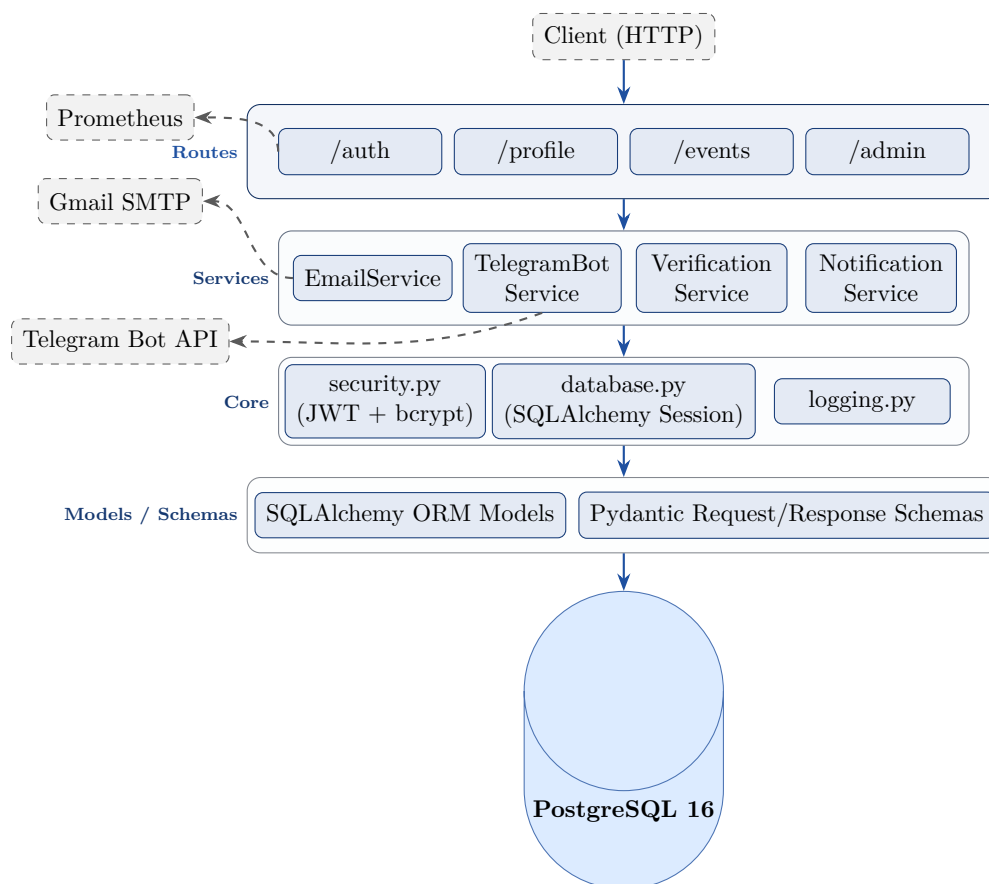


Figure 2: Static View 2 — Backend Component Diagram

Element Catalog

Routes Layer

Contains one file per HTTP operation. Validates inputs using Pydantic schemas, delegates to Services or Core, and returns a Pydantic response model. No business logic lives here.

Services Layer

Encapsulates all external I/O (email sending, Telegram messages, notification dispatching) and non-trivial business logic (verification workflows, OTP generation). Each service is a class with async methods, injected into routes via FastAPI's `Depends()` system.

Core Layer

Provides cross-cutting infrastructure: JWT creation/validation (`security.py`), SQLAlchemy session factory (`database.py`), and structured logging (`logging.py`).

Models / Schemas

ORM models map Python classes to PostgreSQL tables. Pydantic schemas validate incoming request bodies and shape outgoing responses, ensuring malformed data never reaches the database.

PostgreSQL 16

The single authoritative data store. Accessed only through the Core layer's session factory; never directly from Routes.

6.4 Email OTP Authentication Sequence

Figure 3 illustrates the runtime interaction during an email OTP login, the most complex authentication path. The two-phase protocol stores a session token after password verification (step 1–7), then exchanges it for a JWT once the OTP is confirmed (steps 8–12). This prevents JWT issuance before both factors are validated.

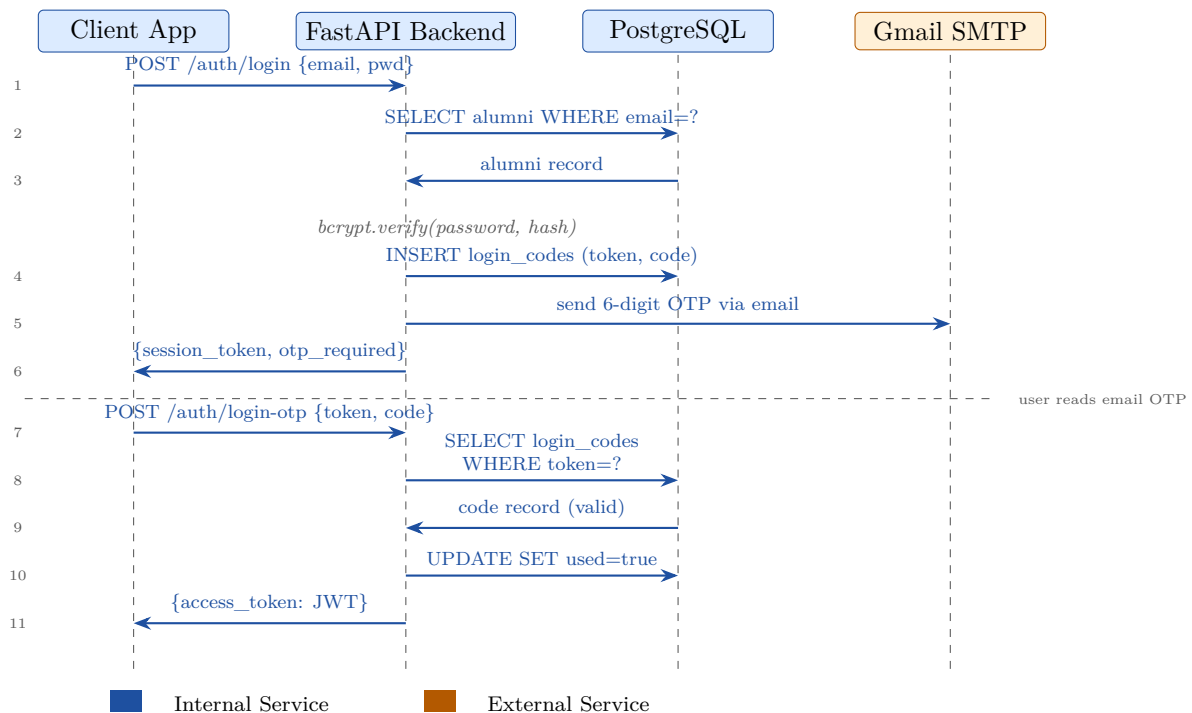


Figure 3: Dynamic View 1 — Email OTP Authentication Sequence

6.5 CI/CD Deployment Pipeline

Figure 4 shows the runtime flow from a code push to a running container on the server. A push to a branch triggers an unbroken automated chain from lint/test through image build, registry push, SSH deployment, and rolling container update. No manual intervention is required for the testing environment; production deployment requires manual approval in the GitHub environment settings. The Blue boxes refer to Github-side steps, while the green boxes refer to server-side steps.

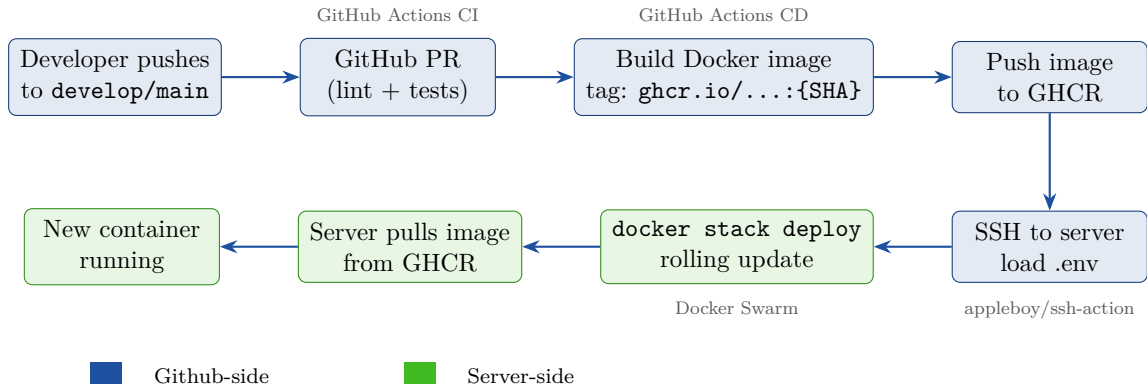


Figure 4: Dynamic View 2 — CI/CD Pipeline

6.6 Deployment Topology

Figure 5 shows the runtime deployment of all services on the single university server. All services share the Docker Swarm overlay network and communicate by container name (Docker DNS). Nginx is the sole component with public ports. PostgreSQL and Prometheus are not publicly routed. The certbot sidecar renews TLS certificates every 12 hours.

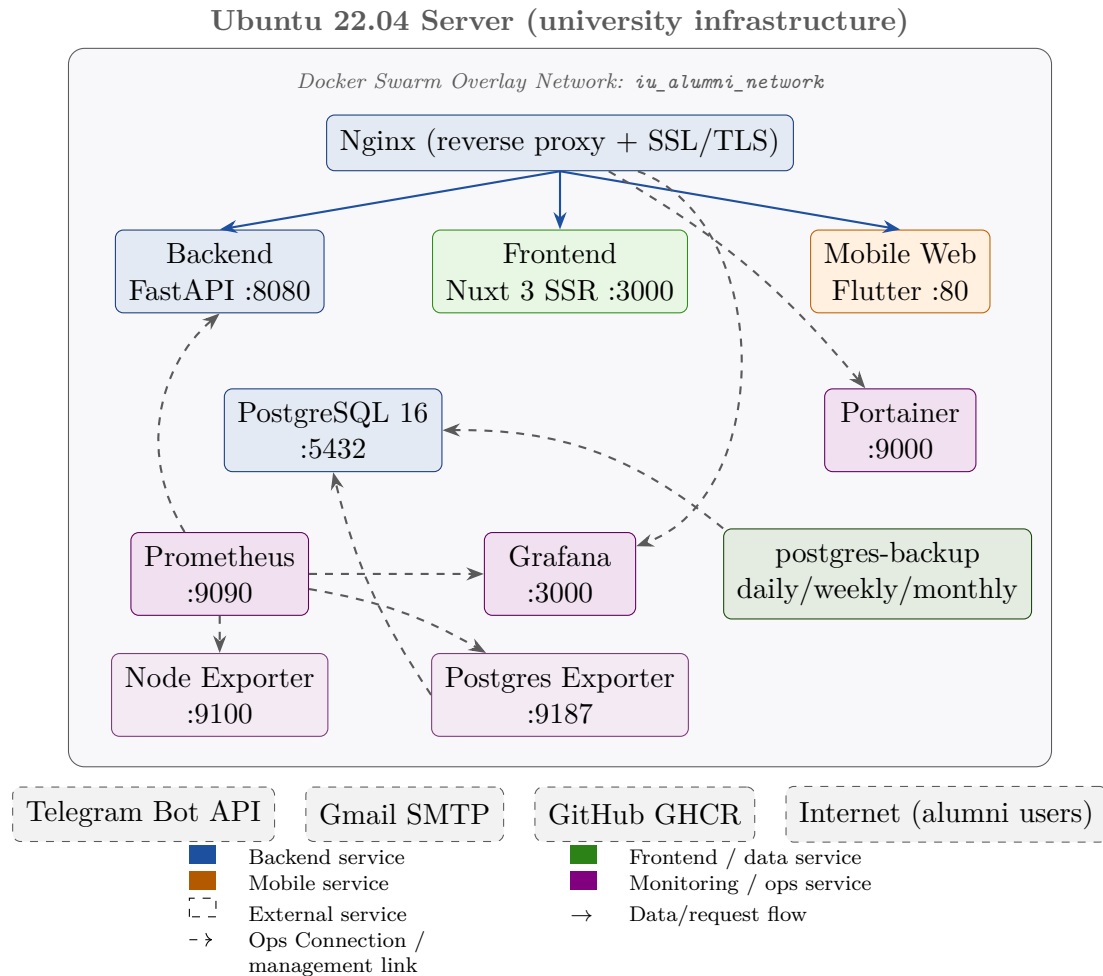


Figure 5: Physical View — Deployment Topology

Element Catalog

Nginx

Single ingress. Performs SSL termination (Let's Encrypt certificates) and routes traffic to backend, frontend, mobile, Grafana, and Portainer by subdomain. Starts independently of application services; routes activate as each service comes up.

Backend (FastAPI :8080)

Serves the REST API. Runs the Telegram long-polling loop as an async background task. Exposes `/metrics` for Prometheus.

Frontend (Nuxt 3 SSR :3000)

Admin portal. Server-side rendering keeps the initial payload small for administrators on typical office networks.

Mobile Web (Flutter :80)

Static Flutter Web build served by Nginx within the container. Loaded by Telegram as a Mini App.

PostgreSQL 16 :5432

Sole database. Pinned to the Swarm manager node to ensure volume persistence. Not publicly exposed.

postgres-backup

Creates daily/weekly/monthly `pg_dump` snapshots into a volume mount. Retention: 7 daily, 4 weekly, 6 monthly.

Prometheus :9090

Scrapes metrics from backend, Node Exporter, and Postgres Exporter every 15 seconds. 30-day TSDB retention. Not publicly exposed.

Grafana :3000

Visualises Prometheus metrics via four pre-provisioned dashboards. Accessible at `grafana.{DOMAIN}`.

Node Exporter :9100

Exports host-level metrics (CPU, memory, disk, network) to Prometheus. Runs in `global` mode (one instance per Swarm node).

Postgres Exporter :9187

Exports PostgreSQL query and connection metrics.

Portainer :9000

Docker Swarm management UI for operators who prefer a GUI.

Certbot

Sidecar process that auto-renews TLS certificates from Let's Encrypt every 12 hours, writing updated certificates to a shared Nginx volume.

6.7 Technology Propositions

Table 4 lists the specific technologies selected and justifies each choice relative to the architectural drivers.

Table 4: Technology Propositions

Category	Technology	Justification
API Framework	FastAPI (Python 3.11)	Native async I/O supports simultaneous HTTP serving and Telegram long-polling (QAS-B). Pydantic integration validates inputs at the boundary (security). Automatic OpenAPI docs accelerate frontend/mobile integration (QAS-G).
Database	PostgreSQL 16	ACID compliance supports QAS-D (data integrity). Rich index types (B-tree, GIN trigram) support efficient pagination and full-text search (QAS-A, QAS-F). Native <code>pg_dump/pg_restore</code> simplifies migration (QAS-M).
ORM / Migrations	SQLAlchemy 2.0 + Alembic	Alembic auto-apply on container start ensures schema and code are always in sync (QAS-G). SQLAlchemy’s type-safe ORM reduces SQL injection risk.
Auth	JWT (python-jose HS256) + bcrypt (passlib)	Stateless JWT eliminates session-store dependency (availability, QAS-B). bcrypt’s adaptive cost factor resists brute-force attacks.
Admin Frontend	Nuxt 3 / Vue 3 / TypeScript	SSR reduces perceived latency for administrators. Pinia provides explicit, testable state. shadcn-nuxt/Tailwind delivers consistent UI without a custom design system (BC1 team size constraint).
Mobile	Flutter (Dart 3.8)	Single codebase compiles to Android APK and Flutter Web (Telegram Mini App), directly satisfying QAS-C and TC2.
State Mgmt (mobile)	BLoC/Cubit	Explicit typed-state transitions are testable (QAS-G) and catch invalid state combinations at compile time.
Orchestration	Docker / Swarm	Zero-overhead alternative to Kubernetes for single-node deployment (BC1, BC2). Provides automatic container restart (QAS-B) and overlay networking for service discovery.
Provisioning	Ansible	Idempotent playbooks make server setup fully reproducible (QAS-M, BC4). The entire server state is expressed as code, so migrating to a new server requires only changing the <code>SERVER_HOST</code> secret.
IaC (GitHub)	Terraform (GitHub provider)	Repository settings, branch protection, and environment secrets are version-controlled. Prevents configuration drift across the four repos (BC4, QAS-G).
CI/CD	GitHub Actions + GHCR	Native to the existing GitHub organisation (TC4). Free for public repos (BC3). GHCR provides authenticated image pulls without an extra registry.
Monitoring	Prometheus + Grafana	Self-hosted, free (BC3). Prometheus pull model fits inside the overlay network (no public exposure). Pre-provisioned Grafana dashboards reduce setup time for future teams (BC4).
SSL/TLS	Let’s Encrypt + Certbot	Free, widely trusted CA (BC3). 90-day certificates auto-renewed every 12 hours by the Certbot sidecar, requiring no manual intervention (BC4).

6.8 Driver Traceability Matrix

Table 5 traces each business goal through its quality attribute driver and design decisions to the implementing components.

Table 5: Driver Traceability Matrix

Business Goal	Driver	Design Decision	Implementing Component(s)
Migrate with minimal downtime	QAS-M	D1.1 (GitOps deployment), D1.3 (pre-migration backup)	Ansible playbooks, <code>deploy.sh</code> , <code>postgres-backup-local</code>
Preserve all alumni data	QAS-D	D1.3 (backup), D5.2 (tiered backups), D5.3 (Alembic auto-apply)	<code>postgres-backup</code> , Alembic migrations
Fast user experience	QAS-A	D2.3 (cursor pagination), D2.4 (DB indexes)	Backend list routes, PostgreSQL indexes
Cross-platform access	QAS-C	D3.1 (Flutter), D3.2 (3-layer + Cubit)	<code>iu-alumni-mobile</code> (Android + Web build)
Reliable authentication	QAS-B	D2.2 (stateless JWT), D4.1 (multi-method auth), D5.1 (monitoring)	Auth routes, <code>security.py</code> , Prometheus alerts
Maintain for future teams	QAS-G, BC4	D2.1 (layered arch), D3.2 (BLoC/Cubit), D5.3 (Alembic)	All layered components; documented deployment scripts
Zero infrastructure cost	BC3	D1.2 (Docker Swarm), D5.1 (self-hosted monitoring)	Docker Swarm, Prometheus, Grafana, Let's Encrypt
Map feature performance	QAS-F	D2.4 (composite index on <code>show_location</code>), cursor pagination	<code>GET /profile/map</code> endpoint, <code>cities</code> table index

7 Risk Mitigation and Alternative Decisions

We identify three architectural decisions that carry significant risk or controversy. For each we describe what could go wrong, an alternative approach, and the trade-offs of switching.

7.1 Risk 1 — Single-Server Deployment (Single Point of Failure)

Decision at risk: D1.2 — Docker Swarm on a Single Node

Decision recap. All application, database, and monitoring services run on one university server inside a single-node Docker Swarm cluster. Nginx, PostgreSQL, the backend, and all monitoring containers share this one machine.

What could go wrong. Hardware failure (disk, NIC, power supply) or a kernel panic takes down the entire platform instantly. There is no standby replica. Even with Docker Swarm's restart policy, hardware-level failures cannot be recovered by software. For the alumni database specifically, if the disk fails between backup windows, up to 24 hours of data could be lost. An extended hardware failure during peak season (e.g., an annual alumni event) would be highly visible and damaging to trust.

Alternative approach. Deploy a two-node Docker Swarm with PostgreSQL replication. Option A: use PostgreSQL streaming replication with a hot standby on the second node; promote on failure. Option B: use a managed cloud PostgreSQL service (e.g., Supabase, Neon) for the data tier while keeping the application servers on university infrastructure. Either approach breaks the single point of failure for data.

Trade-offs of switching.

- Requires a second server or a paid cloud database, conflicting with BC2 and BC3.
- PostgreSQL streaming replication requires Swarm placement constraints and a failover automation script, increasing operational complexity (contradicts BC1 and BC4).

- Managed cloud databases cost money and introduce a data-residency question (where is alumni data stored?).
- **Decision retained because:** The university server is a institutional-grade machine with professional IT management; the risk of unrecoverable hardware failure is lower than typical commodity cloud VMs. Tiered daily backups (D5.2) and a documented restore procedure (`scripts/restore-db.sh`) bound the data-loss window to ≤ 24 hours. This was judged acceptable for a university social network at this scale.

7.2 Risk 2 — Long-Lived JWT Tokens (1-Year Expiry)

Decision at risk: D4.3 — JWT Tokens with 1-Year Expiry

Decision recap. All issued JWTs carry a one-year expiry. There is no refresh-token mechanism. Token revocation requires changing the `SECRET_KEY`, which invalidates all active sessions globally.

What could go wrong. If a user's device is stolen or an account is compromised, the attacker holds a valid JWT for up to one year. The only mitigation is banning the account via the admin portal (which prevents API access by checking `is_banned` on every authenticated request), but the token itself remains cryptographically valid. A mass credential leak (e.g., the JWT secret is exposed) would require emergency key rotation, logging out every user simultaneously — a significant operational incident.

Alternative approach. Introduce a short-lived access token (15-minute expiry) paired with a long-lived refresh token (30-day expiry) stored in the database. On access token expiry, the client uses the refresh token to obtain a new pair. Refresh tokens can be revoked per-user by deleting the DB record, enabling surgical session invalidation without affecting other users.

Trade-offs of switching.

- Requires a new `refresh_tokens` table, two new API endpoints (POST `/auth/refresh`, POST `/auth/logout`), and updated token-refresh logic in both mobile and frontend clients.
- Mobile clients must handle 401 responses by transparently refreshing and retrying; silent failure is unacceptable but the implementation is non-trivial.
- Short tokens improve security at the cost of more frequent database reads (revocation checks) and slightly more complex client-side token management.
- **Decision retained for now because:** The platform stores no financial or highly-sensitive data. Account banning (already implemented) provides an effective incident-response tool. The refresh-token pattern is documented as a recommended future improvement in the project handover notes, with a clear migration path that does not break existing tokens.

7.3 Risk 3 — Flutter Technology Risk (Team Inexperience)

Decision at risk: D3.1 — Flutter for Cross-Platform Mobile

Decision recap. The mobile client is built in Flutter/Dart, compiling to both Android APK and Flutter Web (Telegram Mini App). No team member had prior Flutter or Dart experience before the project began.

What could go wrong. A team without Flutter expertise produces code more slowly, makes non-idiomatic design choices that accumulate as technical debt, and is more likely to misuse framework primitives (e.g., rebuilding widget subtrees on every state change, incorrect async handling in Cubits). Hidden bugs are harder to detect when no reviewer has the domain knowledge to spot them. The learning curve directly contributed to implementation velocity loss already documented as Risk R-07 in the project risk register.

Alternative considered: full rewrite in React or Vue. Given that the team has React and Vue experience from the frontend work, a full rewrite of the Flutter codebase in one of these frameworks was evaluated as a way to eliminate the technology-mismatch risk entirely.

Why the rewrite was rejected.

- **Vibe-coding risk outweighs technology-mismatch risk.** A rewrite executed under time pressure, primarily AI-assisted (“vibe coded”) by developers unfamiliar with the target framework’s idiomatic patterns, was judged more likely to introduce subtle and hard-to-detect bugs than the existing, partially-working Flutter code. Generated code tends to be locally plausible but globally inconsistent; without expert review it accumulates silent correctness issues.
- **Existing Flutter code is not throwaway.** The three-layer architecture (D3.2), Either-based error propagation (D3.3), and BLoC/Cubit state model are already implemented and partially tested. Discarding them resets the project to zero at the cost of the quality architecture already established.
- **Cross-platform target would be harder in React/Vue.** The Flutter Web → Telegram Mini App compilation path is a first-class Flutter feature. Replicating the dual-target build (Android APK + Telegram Mini App) in React/Vue requires maintaining two separate build pipelines, reintroducing the complexity that Flutter was chosen to avoid.
- **Constraints BC1 and BC4.** A rewrite consumes team time that is already budgeted for feature delivery. BC4 (student team, limited hours) makes doubling the mobile implementation work infeasible.

Decision retained: extend the Flutter codebase. The accepted mitigation strategy is to continue development on the Flutter codebase while managing the learning-curve risk directly: use established architectural patterns already in place (3-layer, BLoC/Cubit), rely on Dart’s strong static typing and `flutter analyze` to catch errors at compile time, and limit complexity by delivering a well-defined feature scope rather than exploratory feature additions. Implementation will be slower than it would be on a familiar stack, but the delivered code will be correct and maintainable rather than quickly produced but fragile.

Glossary

ADD	Attribute-Driven Design — an iterative architectural design method that uses quality attribute scenarios as the primary design input.
Alembic	A lightweight database migration tool for Python/SQLAlchemy. Produces versioned migration scripts applied automatically on application startup.
BLoC/Cubit	Business Logic Component / Cubit — a Flutter state-management pattern that separates UI from business logic using streams of typed states.
CORS	Cross-Origin Resource Sharing — an HTTP mechanism that allows a web application on one origin to request resources from a different origin.
Cursor pagination	A pagination strategy where the “next page” is identified by an opaque cursor (last-seen record key) rather than an integer offset.
Docker Swarm	Docker’s native container orchestration mode, supporting multi-node clusters with overlay networking and rolling updates.
Fail2ban	A log-parsing daemon that dynamically blocks IPs that show signs of brute-force attacks.
GHCR	GitHub Container Registry — Docker image storage integrated with GitHub, used here for CI/CD image distribution.
GitOps	A practice where all infrastructure configuration is stored in Git and applied automatically, making Git the single source of truth for system state.
JWT	JSON Web Token — a compact, URL-safe token format used for stateless authentication. Signed with HMAC-SHA256 (HS256) in this project.
OTP	One-Time Password — a short-lived numeric code used as a second authentication factor.
Telegram Mini App	A web application loaded inside the Telegram client through an inline keyboard button or bot menu. Receives the user’s Telegram session context.
UFW	Uncomplicated Firewall — a simplified interface to iptables for managing Linux firewall rules.